Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «ИЗБЫТОЧНОЕ КОДИРОВАНИЕ ДАННЫХ • В ИНФОРМАЦИОННЫХ СИСТЕМАХ. КОД ХЕММИНГА. ИТЕРАТИВНЫЕ КОДЫ»

Выполнил:
Студент 4-го курса
Группы АС-65
Колбашко А.В.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга, итеративного кода

Задание 1. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 или 4.

Вариант 9.

```
import random

MIN_DISTANCE = 3
ERROR_COUNT = 1

G_3 = [
    [1, 0, 0, 0, 0, 1, 1],
    [0, 1, 0, 0, 1, 0, 1],
    [0, 0, 1, 0, 1, 1, 0],
    [0, 0, 0, 1, 1, 1, 1]
]

H_3 = [
    [0, 1, 1, 1, 1, 0, 0],
    [1, 0, 1, 1, 0, 1, 0],
    [1, 1, 0, 1, 0, 0, 1]
]

G_4 = [
    [1, 0, 0, 0, 0, 1, 1, 1],
    [0, 1, 0, 0, 1, 0, 1, 1],
    [0, 0, 1, 0, 1, 1, 0, 1],
    [0, 0, 0, 1, 1, 1, 1, 0]
]

H_4 = [
    [0, 1, 1, 1, 1, 0, 0, 0],
    [1, 0, 1, 1, 0, 1, 0, 0],
    [1, 1, 0, 1, 0, 0, 1, 0],
    [1, 1, 1, 1, 1, 1, 1, 1]
]

K = 4

if MIN_DISTANCE == 3:
    G = G_3
    H = H_3
elif MIN_DISTANCE == 4:
    G = G_4
    H = H_4

R = len(H)
N = len(G[0])


def encode_block(Xk):
    Xn = []
    for column in range(N):
        total = 0
        for row in range(K):
            total += Xk[row] * G[row][column]
        Xn.append(total % 2)
    return Xn


def calculate_syndrome(Yn):
    S = []
    for row in H:
        total = 0
```

```
        for i in range(N):
            total += Yn[i] * row[i]
        S.append(total % 2)
    return S


def add_modulo_2(first_vector, second_vector):
    result = []
    for i in range(N):
        result.append((first_vector[i] + second_vector[i]) % 2)
    return result


def create_error_vector():
    En = [0] * N
    positions = random.sample(range(N), ERROR_COUNT)
    for position in positions:
        En[position] = 1
    return En


def find_error_position(S):
    for column in range(N):
        H_column = []
        for row in range(R):
            H_column.append(H[row][column])
        if H_column == S:
            return column
    return None


def correct_word(Yn, S):
    Y_prime = Yn[:]
    position = find_error_position(S)

    if S == [0] * R:
        return Y_prime, position, "No errors detected"

    if MIN_DISTANCE == 3:
        if ERROR_COUNT == 1 and position is not None:
            Y_prime[position] = (Y_prime[position] + 1) % 2
            return Y_prime, position, "Single error corrected"
        return Y_prime, position, "Two errors cannot be detected or corrected"

    if S[-1] == 1 and position is not None:
        Y_prime[position] = (Y_prime[position] + 1) % 2
        return Y_prime, position, "Single error corrected"

    return Y_prime, position, "Two errors detected by the additional parity check
and not corrected"


def bits_to_text(bits):
    result = ""
    for i in range(0, len(bits), 7):
        result += chr(int(bits[i:i + 7], 2))
    return result


with open("text.txt", "r", encoding="ascii") as file:
    text = file.read().rstrip("\n")

bits = ""
for symbol in text:
    bits += format(ord(symbol), "07b")

message_length = len(bits)
padding = (-message_length) % K
bits_with_padding = bits + "0" * padding
```

```
Xk_blocks = []
Xn_words = []
En_vectors = []
Yn_words = []
S_vectors = []
Y_prime_words = []
error_positions = []
statuses = []

for i in range(0, len(bits_with_padding), K):
    Xk = []
    for bit in bits_with_padding[i:i + K]:
        Xk.append(int(bit))

    Xn = encode_block(Xk)
    En = create_error_vector()
    Yn = add_modulo_2(Xn, En)
    S = calculate_syndrome(Yn)
    Y_prime, position, status = correct_word(Yn, S)

    Xk_blocks.append(Xk)
    Xn_words.append(Xn)
    En_vectors.append(En)
    Yn_words.append(Yn)
    S_vectors.append(S)
    Y_prime_words.append(Y_prime)
    error_positions.append(position)
    statuses.append(status)

decoded_bits = ""
for Y_prime in Y_prime_words:
    for bit in Y_prime[:K]:
        decoded_bits += str(bit)

decoded_bits = decoded_bits[:message_length]

print("d_min =", MIN_DISTANCE)
print("k =", K)
print("r =", R)
print("n =", N)
print("Number of random errors in each word:", ERROR_COUNT)
print("Source text:", text)
print("ASCII binary form (7 bits):", bits)
print("Source message length in bits:", message_length)
print("Added zero bits:", padding)
print("Bits with padding:", bits_with_padding)
print("Matrix G:")
for row in G:
    print(row)
print("Matrix H:")
for row in H:
    print(row)

for i in range(len(Xk_blocks)):
    print()
    print("Block", i + 1)
    print("Xk:", Xk_blocks[i])
    print("Xn:", Xn_words[i])
    print("Xr:", Xn_words[i][K:])
    print("En:", En_vectors[i])
    print("Yn:", Yn_words[i])
    print("Yr:", Yn_words[i][K:])
    print("S = H * Yn^T:", S_vectors[i])
    print("Status:", statuses[i])
    if error_positions[i] is not None:
        print("Syndrome position:", error_positions[i] + 1)
    print("Y'n:", Y_prime_words[i])
    print("Yr':", Y_prime_words[i][K:])

print()
```

```
    print("Decoded bits without added zero bits:", decoded_bits)
    if MIN_DISTANCE == 3 and ERROR_COUNT == 2:
        print("Decoded text is not guaranteed after two errors")
    else:
        print("Decoded text:", bits_to_text(decoded_bits))
```

Разработана программа, которая считывает символы из text.txt, переводит их в представление ASCII, кодирует блоки по четыре бита и применяет матрицы G и H. В актуальной версии выбран код с минимальным расстоянием dmin = 3 и одной случайной ошибкой в каждом кодовом слове.

Для сообщения Hi! получено 21 информационный бит. Перед разбиением на блоки добавлены три нулевых бита; при восстановлении они не включаются в исходную последовательность.

```
PS C:\Users\wlksm\OneDrive\Документы\ChatGPT\КИСБ2> python ham.py
d_min = 3
k = 4
r = 3
n = 7
Number of random errors in each word: 1
Source text: Hi!
ASCII binary form (7 bits): 100100011010010100001
Source message length in bits: 21
Added zero bits: 3
Bits with padding: 100100011010010100001000
Matrix G:
[1, 0, 0, 0, 0, 1, 1]
[0, 1, 0, 0, 1, 0, 1]
[0, 0, 1, 0, 1, 1, 0]
[0, 0, 0, 1, 1, 1, 1]
Matrix H:
[0, 1, 1, 1, 1, 0, 0]
[1, 0, 1, 1, 0, 1, 0]
[1, 1, 0, 1, 0, 0, 1]

Block 1
Xk: [1, 0, 0, 1]
Xn: [1, 0, 0, 1, 1, 0, 0]
Xr: [1, 0, 0]
En: [0, 0, 0, 0, 0, 0, 1]
Yn: [1, 0, 0, 1, 1, 0, 1]
Yr: [1, 0, 1]
S = H * Yn^T: [0, 0, 1]
Status: Single error corrected
Syndrome position: 7
Y'n: [1, 0, 0, 1, 1, 0, 0]
Yr': [1, 0, 0]

Block 2
Xk: [0, 0, 0, 1]
Xn: [0, 0, 0, 1, 1, 1, 1]
Xr: [1, 1, 1]
En: [0, 0, 0, 0, 0, 1, 0]
Yn: [0, 0, 0, 1, 1, 0, 1]
Yr: [1, 0, 1]
S = H * Yn^T: [0, 1, 0]
Status: Single error corrected
Syndrome position: 6
Y'n: [0, 0, 0, 1, 1, 1, 1]
Yr': [1, 1, 1]

Block 3
Xk: [1, 0, 1, 0]
Xn: [1, 0, 1, 0, 1, 0, 1]
Xr: [1, 0, 1]
```

```
En: [0, 0, 1, 0, 0, 0, 0]
Yn: [1, 0, 0, 0, 1, 0, 1]
Yr: [1, 0, 1]
S = H * Yn^T: [1, 1, 0]
Status: Single error corrected
Syndrome position: 3
Y'n: [1, 0, 1, 0, 1, 0, 1]
Yr': [1, 0, 1]

Block 4
Xk: [0, 1, 0, 1]
Xn: [0, 1, 0, 1, 0, 1, 0]
Xr: [0, 1, 0]
En: [1, 0, 0, 0, 0, 0, 0]
Yn: [1, 1, 0, 1, 0, 1, 0]
Yr: [0, 1, 0]
S = H * Yn^T: [0, 1, 1]
Status: Single error corrected
Syndrome position: 1
Y'n: [0, 1, 0, 1, 0, 1, 0]
Yr': [0, 1, 0]

Block 5
Xk: [0, 0, 0, 0]
Xn: [0, 0, 0, 0, 0, 0, 0]
Xr: [0, 0, 0]
En: [0, 1, 0, 0, 0, 0, 0]
Yn: [0, 1, 0, 0, 0, 0, 0]
Yr: [0, 0, 0]
S = H * Yn^T: [1, 0, 1]
Status: Single error corrected
Syndrome position: 2
Y'n: [0, 0, 0, 0, 0, 0, 0]
Yr': [0, 0, 0]

Block 6
Xk: [1, 0, 0, 0]
Xn: [1, 0, 0, 0, 0, 1, 1]
Xr: [0, 1, 1]
En: [1, 0, 0, 0, 0, 0, 0]
Yn: [0, 0, 0, 0, 0, 1, 1]
Yr: [0, 1, 1]
S = H * Yn^T: [0, 1, 1]
Status: Single error corrected
Syndrome position: 1
Y'n: [1, 0, 0, 0, 0, 1, 1]
Yr': [0, 1, 1]

Decoded bits without added zero bits: 100100011010010100001
Decoded text: Hi!
```

Задание 2. Итеративный код

Во второй части реализован итеративный код. В текущем запуске используется трёхмерная конфигурация 4 × 4 × 2: 32 информационных бита расположены в двух матрицах 4 × 4. Применяются пять групп проверок: горизонтальные, вертикальные, две диагональные и проверки по оси глубины.

Программа строит проверочные биты, формирует кодовое слово Xn, вносит две случайные ошибки, вычисляет группы синдромов и пытается последовательно устранить однозначно определяемые искажения. Для

количественной оценки выполнено 100 независимых испытаний.

```
import random

CONFIGURATION = 4
PARITY_GROUPS = 5
ERROR_COUNT = 2
TRIALS = 100

INFORMATION_WORD = "10110010100101101100010111001010"

if CONFIGURATION == 1:
    K1 = 4
    K2 = 8
    Z = 0
elif CONFIGURATION == 2:
    K1 = 2
    K2 = 16
    Z = 0
elif CONFIGURATION == 3:
    K1 = 8
    K2 = 2
    Z = 2
else:
    K1 = 4
    K2 = 4
    Z = 2

if Z == 0:
    DIMENSIONS = 2
else:
    DIMENSIONS = 3

K = K1 * K2
if DIMENSIONS == 3:
    K = K * Z

DIAGONAL_COUNT = max(K1, K2)


def parity(bits):
    total = 0
    for bit in bits:
        total += bit
    return total % 2


def create_data(bits):
    data = []
    position = 0

    if DIMENSIONS == 2:
        plane = []
        for row in range(K1):
            line = []
            for column in range(K2):
                line.append(int(bits[position]))
                position += 1
            plane.append(line)
        data.append(plane)
    else:
        for layer in range(Z):
            plane = []
            for row in range(K1):
                line = []
                for column in range(K2):
                    line.append(int(bits[position]))
                    position += 1
                plane.append(line)
            data.append(plane)

    return data
```

```
def data_to_bits(data):
    bits = []
    for layer in data:
        for row in layer:
            for bit in row:
                bits.append(bit)
    return bits


def group_names():
    names = ["Horizontal", "Vertical"]

    if DIMENSIONS == 2:
        if PARITY_GROUPS == 3:
            names.append("Xhv")
    else:
        if PARITY_GROUPS >= 3:
            names.append("Diagonal_1")
        if PARITY_GROUPS >= 4:
            names.append("Diagonal_2")
        if PARITY_GROUPS >= 5:
            names.append("Depth")

    return names


def calculate_parities(data):
    names = group_names()
    groups = []

    horizontal = []
    for layer in range(len(data)):
        for row in range(K1):
            horizontal.append(parity(data[layer][row]))
    groups.append(horizontal)

    vertical = []
    for layer in range(len(data)):
        for column in range(K2):
            bits = []
            for row in range(K1):
                bits.append(data[layer][row][column])
            vertical.append(parity(bits))
    groups.append(vertical)

    if DIMENSIONS == 2 and PARITY_GROUPS == 3:
        all_bits = data_to_bits(data)
        for group in groups:
            for bit in group:
                all_bits.append(bit)
        groups.append([parity(all_bits)])

    if DIMENSIONS == 3 and PARITY_GROUPS >= 3:
        diagonal_1 = []
        for layer in range(Z):
            for diagonal in range(DIAGONAL_COUNT):
                bits = []
                for row in range(K1):
                    for column in range(K2):
                        if (row + column) % DIAGONAL_COUNT == diagonal:
                            bits.append(data[layer][row][column])
                diagonal_1.append(parity(bits))
        groups.append(diagonal_1)

    if DIMENSIONS == 3 and PARITY_GROUPS >= 4:
        diagonal_2 = []
        for layer in range(Z):
            for diagonal in range(DIAGONAL_COUNT):
                bits = []
                for row in range(K1):
                    for column in range(K2):
                        if (row - column) % DIAGONAL_COUNT == diagonal:
                            bits.append(data[layer][row][column])
                diagonal_2.append(parity(bits))
        groups.append(diagonal_2)
```

```
    if DIMENSIONS == 3 and PARITY_GROUPS >= 5:
        depth = []
        for row in range(K1):
            for column in range(K2):
                bits = []
                for layer in range(Z):
                    bits.append(data[layer][row][column])
                depth.append(parity(bits))
        groups.append(depth)

    return names, groups


def make_codeword(data):
    names, groups = calculate_parities(data)
    Xn = data_to_bits(data)
    for group in groups:
        for bit in group:
            Xn.append(bit)
    return names, groups, Xn


def split_codeword(word, names, group_lengths):
    data_bits = word[:K]
    data = create_data("".join(str(bit) for bit in data_bits))
    received_groups = []
    position = K

    for length in group_lengths:
        group = []
        for i in range(length):
            group.append(word[position])
            position += 1
        received_groups.append(group)

    return data, received_groups


def calculate_syndrome(data, received_groups):
    names, calculated_groups = calculate_parities(data)
    S = []

    for group_number in range(len(names)):
        group_syndrome = []
        for i in range(len(calculated_groups[group_number])):
            group_syndrome.append(
                (calculated_groups[group_number][i] +
received_groups[group_number][i]) % 2
            )
        S.append(group_syndrome)

    return S


def syndrome_is_zero(S):
    for group in S:
        for bit in group:
            if bit == 1:
                return False
    return True


def index_in_group(group_name, layer, row, column):
    if group_name == "Horizontal":
        return layer * K1 + row
    if group_name == "Vertical":
        return layer * K2 + column
    if group_name == "Diagonal_1":
        return layer * DIAGONAL_COUNT + (row + column) % DIAGONAL_COUNT
    if group_name == "Diagonal_2":
        return layer * DIAGONAL_COUNT + (row - column) % DIAGONAL_COUNT
    if group_name == "Depth":
        return row * K2 + column
    return 0
```

```
def position_name(position, names, group_lengths):
    if position < K:
        if DIMENSIONS == 2:
            row = position // K2
            column = position % K2
            return "Xk[" + str(row + 1) + "][" + str(column + 1) + "]"

        layer = position // (K1 * K2)
        remainder = position % (K1 * K2)
        row = remainder // K2
        column = remainder % K2
        return (
            "Xk["
            + str(layer + 1)
            + "]["
            + str(row + 1)
            + "]["
            + str(column + 1)
            + "]"
        )

    position -= K
    for group_number in range(len(names)):
        if position < group_lengths[group_number]:
            return names[group_number] + "[" + str(position + 1) + "]"
        position -= group_lengths[group_number]

    return ""


def find_data_candidates(S, names):
    candidates = []

    for layer in range(len_data_layers):
        for row in range(K1):
            for column in range(K2):
                is_candidate = True
                for group_number in range(len(names)):
                    group_index = index_in_group(names[group_number], layer, row,
column)
                    if S[group_number][group_index] != 1:
                        is_candidate = False
                if is_candidate:
                    candidates.append((layer, row, column))

    return candidates


def correct_codeword(Yn, names, group_lengths):
    data, received_groups = split_codeword(Yn, names, group_lengths)
    corrected_positions = []

    for step in range(ERROR_COUNT * 2):
        S = calculate_syndrome(data, received_groups)

        if syndrome_is_zero(S):
            break

        candidates = find_data_candidates(S, names)

        if len(candidates) == 1:
            layer, row, column = candidates[0]
            data[layer][row][column] = (data[layer][row][column] + 1) % 2
            data_position = layer * K1 * K2 + row * K2 + column
            corrected_positions.append(
                position_name(data_position, names, group_lengths)
            )
        else:
            parity_position_count = 0
            parity_group = 0
            parity_index = 0

            for group_number in range(len(S)):
                for i in range(len(S[group_number])):
                    if S[group_number][i] == 1:
                        parity_position_count += 1
```

```
                        parity_group = group_number
                        parity_index = i

            if parity_position_count == 1:
                received_groups[parity_group][parity_index] = (
                    received_groups[parity_group][parity_index] + 1
                ) % 2
                offset = K
                for i in range(parity_group):
                    offset += group_lengths[i]
                corrected_positions.append(
                    position_name(offset + parity_index, names, group_lengths)
                )
            else:
                break

    Y_prime = data_to_bits(data)
    for group in received_groups:
        for bit in group:
            Y_prime.append(bit)

    S_final = calculate_syndrome(data, received_groups)
    return Y_prime, S_final, corrected_positions


def create_received_word(Xn):
    En = [0] * len(Xn)
    positions = random.sample(range(len(Xn)), ERROR_COUNT)

    for position in positions:
        En[position] = 1

    Yn = []
    for i in range(len(Xn)):
        Yn.append((Xn[i] + En[i]) % 2)

    return En, Yn, positions


def print_data(data):
    for layer in range(len(data)):
        if DIMENSIONS == 3:
            print("Layer", layer + 1)
        for row in data[layer]:
            print(row)


data = create_data(INFORMATION_WORD)
names, Xr_groups, Xn = make_codeword(data)
group_lengths = []
for group in Xr_groups:
    group_lengths.append(len(group))

len_data_layers = len(data)

En, Yn, error_positions = create_received_word(Xn)
received_data, received_groups = split_codeword(Yn, names, group_lengths)
S = calculate_syndrome(received_data, received_groups)
Y_prime, S_prime, corrected_positions = correct_codeword(Yn, names, group_lengths)

print("Configuration:", CONFIGURATION)
print("Dimensions:", DIMENSIONS)
print("k1 =", K1)
print("k2 =", K2)
print("z =", Z)
print("k =", K)
print("r =", len(Xn) - K)
print("n =", len(Xn))
print("Parity groups:", PARITY_GROUPS)
print("Error count:", ERROR_COUNT)
print("Source information word Xk:", INFORMATION_WORD)
print("Information matrix:")
print_data(data)
print("Parity groups Xr:")
for i in range(len(names)):
    print(names[i] + ":", Xr_groups[i])
```

```
print("Codeword Xn:", "".join(str(bit) for bit in Xn))
print("Error vector En:", "".join(str(bit) for bit in En))
print("Injected error positions:")
for position in error_positions:
    print(position_name(position, names, group_lengths))
print("Received word Yn:", "".join(str(bit) for bit in Yn))
print("Received information matrix:")
print_data(received_data)
print("Syndrome groups:")
for i in range(len(names)):
    print(names[i] + ":", S[i])
print("Corrected positions:")
for position in corrected_positions:
    print(position)
print("Corrected word Y'n:", "".join(str(bit) for bit in Y_prime))
print("Final syndrome:")
for i in range(len(names)):
    print(names[i] + ":", S_prime[i])
print("Xn == Y'n:", Xn == Y_prime)

N1 = TRIALS
N2 = 0
N3 = 0

for trial in range(TRIALS):
    En_trial, Yn_trial, error_positions_trial = create_received_word(Xn)
    Y_prime_trial, S_prime_trial, corrected_positions_trial = correct_codeword(
        Yn_trial, names, group_lengths
    )

    actual_positions = []
    for position in error_positions_trial:
        actual_positions.append(position_name(position, names, group_lengths))

    if sorted(actual_positions) == sorted(corrected_positions_trial):
        N2 += 1

    if Xn == Y_prime_trial:
        N3 += 1

print("Analysis for", ERROR_COUNT, "random errors:")
print("N1 =", N1)
print("N2 =", N2)
print("N3 =", N3)
print("N2/N1 =", N2 / N1)
print("N3/N1 =", N3 / N1)


PS C:\Users\wlksm\OneDrive\Документы\ChatGPT\КИСБ2> python iterative.py
Configuration: 4
Dimensions: 3
k1 = 4
k2 = 4
z = 2
k = 32
r = 48
n = 80
Parity groups: 5
Error count: 2
Source information word Xk: 10110010100101101100010111001010
Information matrix:
Layer 1
[1, 0, 1, 1]
[0, 0, 1, 0]
[1, 0, 0, 1]
[0, 1, 1, 0]
Layer 2
[1, 1, 0, 0]
[0, 1, 0, 1]
[1, 1, 0, 0]
[1, 0, 1, 0]
Parity groups Xr:
Horizontal: [1, 1, 0, 0, 0, 0, 0, 0]
Vertical: [0, 1, 1, 0, 1, 1, 1, 1]
Diagonal_1: [0, 0, 0, 0, 0, 0, 0, 0]
Diagonal_2: [1, 0, 1, 0, 0, 0, 0, 0]
```

```
Depth: [0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0]
Codeword Xn:
1011001010010110110001011100101011000000011011110000000010100000011101110101110 0
Error vector En:
1000000000000000000000000000000000000000000000000000000010000000000000000000000 0
Injected error positions:
Xk[1][1][1]
Diagonal_2[2]
Received word Yn:
0011001010010110110001011100101011000000011011110000000011100000011101110101110 0
Received information matrix:
Layer 1
[0, 0, 1, 1]
[0, 0, 1, 0]
[1, 0, 0, 1]
[0, 1, 1, 0]
Layer 2
[1, 1, 0, 0]
[0, 1, 0, 1]
[1, 1, 0, 0]
[1, 0, 1, 0]
Syndrome groups:
Horizontal: [1, 0, 0, 0, 0, 0, 0, 0]
Vertical: [1, 0, 0, 0, 0, 0, 0, 0]
Diagonal_1: [1, 0, 0, 0, 0, 0, 0, 0]
Diagonal_2: [1, 1, 0, 0, 0, 0, 0, 0]
Depth: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Corrected positions:
Xk[1][1][1]
Diagonal_2[2]
Corrected word Y'n:
1011001010010110110001011100101011000000011011110000000010100000011101110101110 0
Final syndrome:
Horizontal: [0, 0, 0, 0, 0, 0, 0, 0]
Vertical: [0, 0, 0, 0, 0, 0, 0, 0]
Diagonal_1: [0, 0, 0, 0, 0, 0, 0, 0]
Diagonal_2: [0, 0, 0, 0, 0, 0, 0, 0]
Depth: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Xn == Y'n: True
Analysis for 2 random errors:
N1 = 100
N2 = 48
N3 = 48
N2/N1 = 0.48
N3/N1 = 0.48
```

Вывод : В ходе работы были реализованы два способа избыточного кодирования. Код Хемминга с dmin = 3 корректно определил и исправил одиночные ошибки во всех блоках, после чего исходное сообщение было восстановлено. d=4 же обнаружил две ошибки но не исправил их.

Итеративный код был наиболее ёмко проверен на трёхмерной структуре 4 × 4 × 2 с пятью группами паритетов. При двух случайных ошибках программа восстановила исходное кодовое слово в 48 из 100 испытаний. В остальных случаях синдромы фиксировали нарушение проверок, но совокупность признаков не позволяла однозначно выбрать корректируемые позиции. Полученный результат показывает, что увеличение числа независимых паритетных групп повышает информативность контроля, однако не гарантирует исправление каждой комбинации нескольких ошибок.